Kinematic Restitution Balancing (KRB)

A Per-Constraint Energy Audit for Velocity-Level Impulse Solvers

Joshua Sideris
Gear3Games
https://gearbox2d.com
josh.sideris@gmail.com

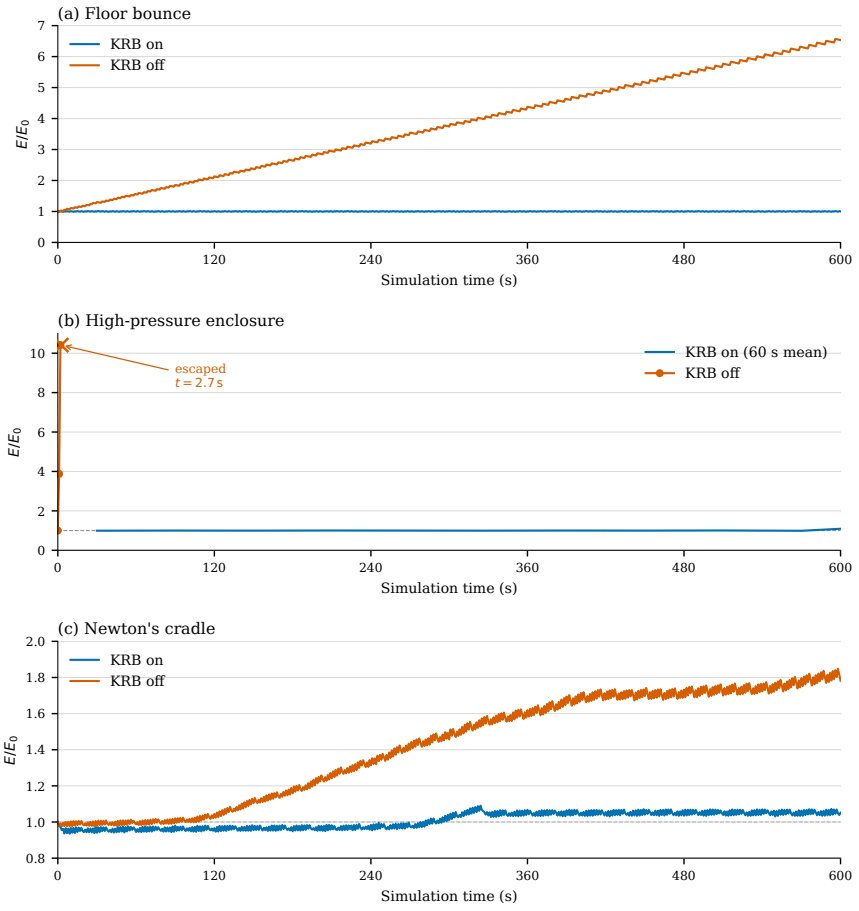


Figure 1. Mechanical energy ratio E/E_0 over $600\,\mathrm{s}$. (a) Elastic floor contact. (b) High-rate enclosure; KRB on is a $60\,\mathrm{s}$ windowed mean, KRB off raw $1\,\mathrm{Hz}$ samples to tunneling ($t = 2.7\,\mathrm{s}$). (c) Newton's cradle.

Abstract

Velocity-level sequential-impulse solvers gain energy from two analytic leaks: restitution and joint bias see the post-force velocity, and Baumgarte displacement does potential-energy work without a matching kinetic-energy tax. Kinematic Restitution Balancing (KRB) is a per-constraint `preSolve` audit of those leaks: Component A subtracts the force increment from the impact velocity, and Component B adjusts launch or bias speed for the expected correction Δh. A compile-time on/off ablation in Gearbox2D keeps a single elastic floor bounce at $E/E_0 \approx 1$ over $600\,\mathrm{s}$ (KRB off finishes at 6.53) and holds a high-rate enclosure in-box; a Newton's cradle remains a bounded offset, not an invariant. The method adds no solver passes.

1. Introduction

Velocity-level sequential-impulse (SI) solvers [Catto 2005] are the standard real-time rigid-body approach: constraints are solved after explicit force integration with a fixed iteration count. KRB is a drop-in *preSolve* energy audit for that pipeline. It does not add solver passes, replace PGS, or switch to position-based dynamics.

Bender, Erleben, and Trinkle survey interactive rigid-body practice [Bender et al. 2014]. Baumgarte stabilization [Baumgarte 1972] and split impulse / NGS position passes [Catto 2014, 2024] hide bounce-from-correction but do not tax PE work of the remaining Δh; TGS, soft constraints, and XPBD [Catto 2011; NVIDIA Corporation 2024; Macklin et al. 2016; Müller et al. 2007] damp or sidestep velocity-level bias at different cost. Variational integrators [Hairer et al. 2006] bound unconstrained drift; KRB is not one of those. Among the SI variants cited here, none apply an analytic PE/KE balance to expected Δh for both contacts and distance joints inside a single-pass impulse solver.

Two analytic sources inject artificial mechanical energy during constraint resolution.

Force-integration drift. With symplectic Euler integration, velocity is updated before the constraint solve:

$$v_{\text{solver}} = v_t + a_{\text{ext}} \cdot \Delta t. \tag{1}$$

Restitution and joint bias see v_{solver} instead of the true relative velocity v_{impact}. For a perfect bounce ($e = 1$), the launch velocity gains $a_{\text{ext}} \cdot \Delta t$ extra speed per frame. For a joint at rest, the solver sees gravity-induced velocity and fights it every frame at the velocity level.

Potential-energy shift from position correction. Baumgarte stabilization [Baumgarte 1972] feeds constraint error back as a velocity bias $v_{\text{bias}} = \beta C/\Delta t$, displacing bodies by Δh to resolve penetration or joint drift. That displacement increases potential

energy (mgh) without reducing kinetic energy. The added PE converts to KE on later frames, causing runaway energy growth.

KRB applies that PE/KE balance to expected Δh at constraint setup for contacts and distance joints. The audit is per-constraint; coupled graphs can still offset (Section 6).

2. Method

KRB performs two independent corrections during constraint `preSolve`.

2.1. Component A: Force velocity compensation

With symplectic Euler, integration updates velocity before the constraint solve (Equation 1), so restitution and joint bias see v_{solver} rather than the pre-force relative velocity. Store the per-body force increment $v_{\text{force}} = a_{\text{ext}} \cdot \Delta t$ during integration and subtract it from the relative velocity seen at setup. That recovers the true impact normal velocity before restitution or bias is applied:

$$v_{\text{impact}} = v_{\text{relative}} - (v_{\text{force},B} - v_{\text{force},A}). \tag{2}$$

The correction is independent of the bias method.

2.2. Component B: Kinematic energy balancing

After Component A, tax (or credit) the launch speed for the potential-energy work of this step's expected correction Δh (Section 2.3). For a dynamic body on a static floor, center-of-mass motion along $\mathbf{n}$, free-flight rewind is $v^2_{\text{impact}} = v^2_{\text{surf}} - 2(\mathbf{a} \cdot \mathbf{n})\Delta h$. Equivalently, $\Delta E_p = -m(\mathbf{a} \cdot \mathbf{n})\Delta h$ taken from $\frac{1}{2}mv^2$ gives $\Delta(v^2) = 2(\mathbf{a} \cdot \mathbf{n})\Delta h$, so m cancels. The implementation uses the relative normal acceleration $\mathbf{a}_{\text{rel}} = \mathbf{a}_B - \mathbf{a}_A$ (Listing 1, `forceVn`$/\Delta t$):

$$v_{\text{surf}} = \sqrt{\max\big(0,\, v^2_{\text{impact}} + 2(\mathbf{a}_{\text{rel}} \cdot \mathbf{n})\Delta h\big)}, \qquad v_{\text{final}} = e\, v_{\text{surf}}. \tag{3}$$

Ground collisions ($\mathbf{a}_{\text{rel}} \cdot \mathbf{n} < 0$) tax launch speed; ceiling collisions credit it. If both bodies share the same $\mathbf{a}_{\text{ext}}$, $\mathbf{a}_{\text{rel}} = \mathbf{0}$ and Component B is a no-op. Rotation, tangent motion, and impulse work are outside this identity (Section 6).

2.3. Effective displacement prediction

Contacts and distance joints share Equation (3); they do not share Δh.

For a bouncing contact, the kinematic bounce $v_{\text{launch}} = -e \cdot v_{\text{impact}}$ clears some overlap before the position pass. After slop s,

$$d_{\text{eff}} = \max\big(0,\, (d - s) - v_{\text{launch}}\Delta t\big), \tag{4}$$

$$\Delta h = \min(d_{\text{eff}}, \Delta h_{\max}) \cdot \Gamma, \tag{5}$$

with $\Gamma = 1 - (1 - \beta)^n$ over n position iterations. Distance joints use the bias travel $\Delta h_{\text{joint}} = \beta C$ (Section 3.2), not Γ.

These Δh values are explicit predictors of Baumgarte travel, not a measurement of the later position pass. Using the pre-tax v_{launch} keeps Δh closed-form; a post-tax v_{final} would make it implicit. On a ground tax the predictor is then slightly low—same sign as the original leak. Γ is the isolated geometric series, not a fit to coupled NGS. If $v_{\text{launch}}\Delta t$ already clears $(d - s)$, $d_{\text{eff}} = 0$ and there is no loop.

3. Constraint application

3.1. Contacts and speculative gaps

For physical overlap ($d > 0$), both Component A and Component B are required. Speculative contacts (created before overlap, $d < 0$) use Component A only since $\Delta h = 0$. Apply restitution bias when enabled and velocity exceeds a threshold or a speculative gap closes too fast. Resting contacts must not receive an $e = 1$ launch bias.

3.2. Distance joints

Distance joints must apply Component B to avoid energy leak when Baumgarte correction does work against gravity, but cap it so a resting joint is not paralyzed. The audited correction is

$$v_{\text{bias_actual}} = \sqrt{\max\big(0,\, v_{\text{bias}}^2 - 2(\mathbf{a}_{\text{rel}} \cdot \mathbf{n})(\beta C)\big)}, \tag{6}$$

where $v_{\text{bias}} = \beta C/\Delta t$ and $\Delta h_{\text{joint}} = \beta C$. If kinetic energy cannot pay the PE tax, the joint retains a microscopic Baumgarte sag—bounded audit rather than infinite stiffness.

4. Implementation

KRB runs at contact and distance-joint `preSolve`, before the velocity iteration loop. Prerequisites: symplectic Euler with per-body `forceVelocity` ($a_{\text{ext}}\Delta t$), Baumgarte position correction with factor β, position iteration count n, and penetration slop. Store $v_{\text{force}} = a_{\text{ext}}\Delta t$ per dynamic body at integrate (before the constraint solve); here a_{ext} is gravity plus external force before linear damping; the store is zero for sleeping or infinite-mass bodies. Full listings in supplement; Listings 1 and 2 show the measured recipes for contacts and distance joints.

4.1. Contact and joint setup

```
forceVn = dot(bodyB.forceVelocity - bodyA.forceVelocity, n)
relativeVn = vn - forceVn
```

```
vBounce = -e * relativeVn
shouldBounce = enableRestitution &&
  (relativeVn < -RESTITUTION_THRESHOLD ||
   (depth < 0 && relativeVn < depth / dt))
if (shouldBounce) {
  if (depth > 0) {
    depthAfterVelocity = max(0, (depth - slop) - vBounce * dt)
    cumulativeFactor = 1 - pow(1 - beta, n)
    expectedDisplacement =
      min(depthAfterVelocity, maxPosCorr) * cumulativeFactor
  } else { expectedDisplacement = 0 }
  workTerm = 2 * (forceVn / dt) * expectedDisplacement
  vFinal = e * sqrt(max(0, relativeVn * relativeVn + workTerm))
  bias = (depth < 0) ? -max(vFinal, depth / dt) : -vFinal
} else if (depth < 0) { bias = -depth / dt } else { bias = 0 }
```

Listing 1. Contact `preSolve` bias (Component A + B).

```
forceVn = dot(bodyB.forceVelocity - bodyA.forceVelocity, n)
v_bias_ideal = beta * C / dt
expectedDisplacement = v_bias_ideal * dt
workTerm = 2 * (forceVn / dt) * expectedDisplacement
v_bias_sq = v_bias_ideal * v_bias_ideal
v_bias_actual = sqrt(max(0, v_bias_sq - workTerm))
bias = (v_bias_ideal > 0 ? v_bias_actual : -v_bias_actual) - forceVn
```

Listing 2. Distance-joint `preSolve` bias (Component A + B).

During the velocity solve, $-\mathrm{forceVn}$ is folded into `bias` so `relative_vn` is not double-corrected.

5. Evaluation

Four datasets: A $600\,\mathrm{s}$ energy ablation; D native `world.step()` wall-time; B/C in supplement. Protocol: $dt = 1/60$, $e = 1$, zero friction and damping, sleep off; Datasets A and C use stock `World()` plus `constants.h` (`velocity_iterations`$=8$, `position_iterations`$=3$, $\beta = 0.2$, slop 0.016, restitution threshold 0.01). The ablation switch is full KRB on/off. $E = E_k + E_p$ (including rotational KE); the reported ratio is E/E_0.

Dataset C continues KRB-on traces to $8\,\mathrm{h}$; see supplement for figures and full results. Dataset B places Gearbox2D next to unmodified Box2D-WASM, p2.js, and Matter.js for whole-engine context; see supplement for details.

5.1. Cost

Per dynamic body, KRB stores $\mathrm{forceVelocity}$; each contact or distance-joint `preSolve` adds a dot, optional Γ, a `workTerm`, one `sqrt`, and a bias update. No extra PGS, velocity, or position iterations. Table 2 reports native `g++` on/off wall-time.

Table 1. Dataset A ablation: E/E_0 at $600\,\mathrm{s}$ (compile-time KRB on/off).

Scene	KRB	E/E_0	Win. mean	Outcome
Floor bounce	on	0.999	1.000	bounded
Floor bounce	off	6.53	climbing	runaway
High-pressure circle	on	—	1.00	stayed in box
High-pressure circle	off	—	—	escaped, step 161
Newton's cradle	on	1.05	1.05 after $t = 300$	bounded offset
Newton's cradle	off	1.79	climbing	secular gain

Table 2. Dataset D: native `g++` wall-time per `world.step()` (mean and range, ms/step). Stack is contact-heavy, not Dataset A.

Scene	KRB on	KRB off
Newton's cradle	0.051 (0.050–0.052)	0.052 (0.050–0.053)
Large stack	0.448 (0.445–0.453)	0.426 (0.419–0.432)

Paper scenes sit in timer scatter; the denser stack is about $+5\%$ mean with non-overlapping repeats (whole-step wall time, not isolated `preSolve`).

6. Limitations

KRB is a per-constraint audit, not a coupled one: resolving one constraint can force work on another. In the Newton's cradle this shows as a step from ~ 0.96 to ~ 1.05 near four minutes (Figure 1c); see supplement for 8 h results. The Dataset C enclosure stays in-box but $600\,\mathrm{s}$ windowed E/E_0 drifts $1.006 \to 0.976$ (slow loss, not the KRB-off SI gain).

Energy results use the protocol in Section 5: single-pass SI/PGS, symplectic Euler, contacts and distance joints, $e = 1$, zero friction and damping, sleep off. Friction dissipates, and $e < 1$ skips the bounce tax when `shouldBounce` is false, so either would mask the audit. A coupled-constraint audit, tunneling, ill-conditioned chains, other integrators, hinge joints, and a switch to XPBD or TGS remain future work.

7. Conclusion

KRB is a `preSolve` audit for sequential-impulse solvers: Component A subtracts symplectic-Euler force drift from the impact velocity, and Component B taxes the expected correction Δh. Contact scenes in the Gearbox2D ablation stay bounded; the coupled-constraint gap is in Section 6.

References

BAUMGARTE, J. Stabilization of constraints and integrals of motion in dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 1(1):1–16, 1972. URL: https://doi.org/10.1016/0045-7825(72)90018-7. 2

BENDER, J., ERLEBEN, K., AND TRINKLE, J. Interactive simulation of rigid body dynamics in computer graphics. *Computer Graphics Forum*, 33(1):246–270, 2014. URL: https://doi.org/10.1111/cgf.12272. 2

CATTO, E. Iterative dynamics with temporal coherence. Presented at the Game Developers Conference, San Francisco, CA, 2005. URL: https://box2d.org/files/ErinCatto_IterativeDynamics_GDC2005.pdf. 2

CATTO, E. Soft constraints. Presented at the Game Developers Conference, San Francisco, CA, 2011. URL: https://box2d.org/files/ErinCatto_SoftConstraints_GDC2011.pdf. 2

CATTO, E. Understanding constraints. Presented at the Game Developers Conference, San Francisco, CA, 2014. URL: https://box2d.org/files/ErinCatto_UnderstandingConstraints_GDC2014.pdf. 2

CATTO, E. Solver2D. Box2D blog post, February 2024. URL: https://box2d.org/posts/2024/02/solver2d/. 2

HAIRER, E., LUBICH, C., AND WANNER, G. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, volume 31 of *Springer Series in Computational Mathematics*. Springer-Verlag, Berlin, Germany, 2 edition, 2006. URL: https://doi.org/10.1007/3-540-30666-8. 2

MACKLIN, M., MÜLLER, M., AND CHENTANEZ, N. XPBD: Position-based simulation of compliant constrained dynamics. In *Proc. 9th ACM SIGGRAPH Conference on Motion in Games (MIG)*, pages 49–54, Burlingame, CA, 2016. URL: https://doi.org/10.1145/2994258.2994272. 2

MÜLLER, M., HEIDELBERGER, B., HENNIX, M., AND RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118, 2007. URL: https://doi.org/10.1016/j.jvcir.2007.01.005. 2

NVIDIA CORPORATION. Rigid body dynamics. PhysX SDK 5.4 Documentation, 2024. URL: https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/RigidBodyDynamics.html. 2

Index of Supplemental Materials

The archive `krb-supplement.zip` (ISC) contains: `README.md`; `listings/` (contact, distance-joint, integrate); `data/dataset-a/` (600 s ablation), `data/dataset-b/` (multi-engine), `data/dataset-c/` (8 h continuation), `data/timing/` (native wall-time); `plots/` (redraw scripts); `figures/` (originals). Recapture uses Gearbox2D tag `krb-jcgt-1` at https://github.com/JSideris/Gearbox2D.

Author Contact and License

Joshua Sideris, Gear3Games — josh.sideris@gmail.com, gearbox2d.com. Supplemental code and data in `krb-supplement.zip` are ISC-licensed.

Joshua Sideris, Kinematic Restitution Balancing: A Per-Constraint Energy Audit for Velocity-Level Impulse Solvers, *Journal of Computer Graphics Techniques (JCGT)*, vol. ?, no. ?, 1–1, ????
????

Received:	????-??-??		
Recommended:	????-??-??	Corresponding Editor:	??? ???
Published:	????-??-??	Editor-in-Chief:	Alexander Wilkie

© ???? Joshua Sideris (the Authors).
The Authors provide this document (the Work) under the Creative Commons CC BY-ND 4.0 license available online at http://creativecommons.org/licenses/by-nd/4.0/. The Authors further grant permission for reuse of images and text from the first page of the Work, provided that the reuse is for the purpose of promoting and/or summarizing the Work in scholarly venues and that any reuse is accompanied by a scientific citation to the Work.

